Momo Paradise

Database Design & Implementation for a Multi-Branch Shabu Buffet Chain

ICCS 225 Database Foundations — Term 2025-26 T3

Chaiyanun Sakulsaowapakkul 6681299 Sorawich Pimsen 6681297 Nathanon Chirattithphan 6780756

The Business *all-you-can-eat shabu, many branches, timed sessions*

Momo Paradise sells a time-limited buffet priced per head, with different tiers unlocking premium meats. Everything below must work per branch, in real time:

1

Queues & bookings

Walk-in queues and timed reservations compete for the same tables.

2

Timed sessions

Every table is a ~90-minute session: start, order window, end.

3

Tiered ordering

A Standard table must not be able to order Wagyu — tier rules per dish.

4

Billing & points

Per-head price × guests + extras − coupons, then membership points.

Doing this by hand causes long waits, wrong-tier orders, and billing mistakes

Key Features *two user groups, one database*

CUSTOMER (app)

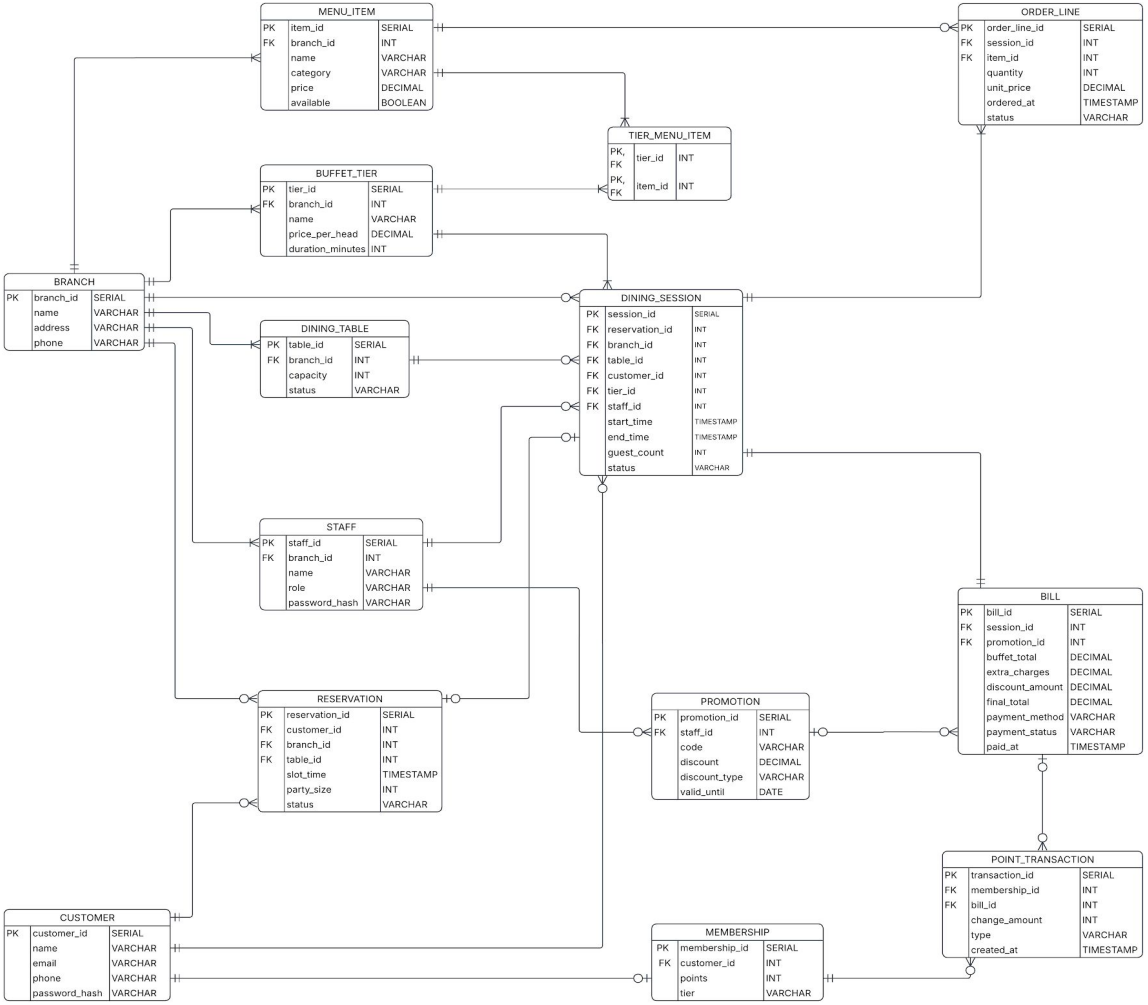
- Register & log in (member account)
- Browse branches & live table availability
- Reserve a slot or join the walk-in queue
- Order dishes during the session — tier-filtered menu
- See the running bill in real time
- Membership: points balance & history

STAFF (back office)

- Seat the queue: assign tables to waiting parties
- Open sessions: table + tier + guest count + timer
- Kitchen view: live unserved orders, mark as served
- Manage menu items & per-branch availability
- Create promotions / coupon codes
- Checkout: bill, discount, points — one transaction

Minor features of the real app (gift cards, ads, …) are out of scope by design.

ER Diagram *14 tables*



Design Highlights

1

Tier ↔ Menu, many-to-many

TIER_MENU_ITEM junction table defines exactly which dishes each buffet tier may order — the signature Momo rule (Standard can't order Wagyu) becomes a single EXISTS check.

2

One RESERVATION shape for queue + booking

slot_time NULL ⇒ walk-in queue entry, otherwise timed booking; table_id stays NULL until staff seat the party. One table, one status machine: reserved / queued / seated / cancelled / no_show.

3

Price snapshot at order time

ORDER_LINE.unit_price copies MENU_ITEM.price when ordered, and BILL stores its computed totals — historical bills stay correct even if menu prices change later.

Design Highlights

1

Tier ↔ Menu, many-to-many

TIER_MENU_ITEM junction table defines which dishes each buffet tier may order — the signature Momo rule (Standard can't order Wagyu) becomes a single EXISTS check in fn_place_order.

2

One RESERVATION shape for queue + booking

slot_time NULL ⇒ walk-in queue entry, otherwise timed booking; table_id stays NULL until staff seat the party. One status machine: reserved / queued / seated / cancelled / no_show.

3

Price snapshot at order time — our one 3NF exception

ORDER_LINE.unit_price copies MENU_ITEM.price when ordered; BILL stores its computed totals. Deliberate denormalization: historical bills stay correct when menu prices change.

Normalization: the schema satisfies 3NF — atomic values, the only composite key (TIER_MENU_ITEM) has no non-key attributes, no transitive dependencies. Highlight 3 is the one exception, on purpose.

Security *three layers, enforced in the database*

1

Passwords: bcrypt in the DB

pgcrypto: crypt(pw, gen_salt('bf')). No plaintext is ever stored or compared; login returns rows only on a correct hash match — and doesn't reveal which field was wrong.

2

SQL injection: parameterized only

Flask has one DB helper, call_fn(): SELECT * FROM fn_x(%s, %s). Arguments are always bound parameters — there is no string-built SQL anywhere in the app.

3

Least privilege: EXECUTE-only role

The app connects as momo_app: REVOKE ALL on tables & sequences, GRANT EXECUTE on fn_* only. Functions are SECURITY DEFINER with a pinned search_path — even a successful injection could only call the same 26 functions.

```
SET ROLE momo_app;  SELECT * FROM customer;   -- ERROR: permission denied
```

Architecture & Tech Stack *thin app, smart database*

Design principle: the app never touches tables. Every read and write goes through a PostgreSQL function — Flask's only DB helper is call_fn(name, args), so business rules are enforced in exactly one place.

Docker Compose
one-command dev environment: db + backend + frontend

deploy.sh
re-runs schema / functions / security against Render

DataGrip + psql
development, testing, example results for the report

28 SQL Functions

one per screen responsibility — the next slides map them to the UI

Auth & Membership

fn_register_customer
fn_login_customer
fn_login_staff
fn_get_session_owner
fn_get_membership
fn_get_point_history

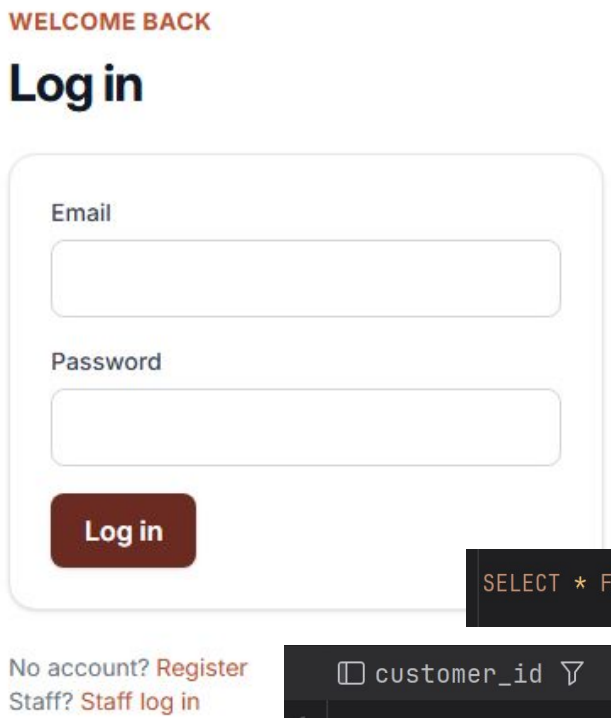
Customer journey

fn_get_branches
fn_get_available_tables
fn_create_reservation
fn_get_tier_menu
fn_place_order
fn_get_session_orders
fn_get_current_bill

Staff operations

fn_get_queue
fn_seat_reservation
fn_open_session
fn_get_menu_items
fn_get_active_sessions
fn_get_tiers
fn_get_kitchen_orders
fn_update_order_status
fn_add_menu_item
fn_update_item_availability
fn_create_promotion
fn_get_promotions
fn_validate_promotion
fn_checkout

Screens → Functions: Sign up & Log in *customer + staff auth*




```
fn_register_customer(name, email, phone, pw)
```

→ returns new customer_id; bcrypt-hashes pw, creates membership row (0 pts)

```
fn_login_customer('mail@example.com', 'pw')
```

→ correct → the customer row; wrong → 0 rows (app shows failed login)

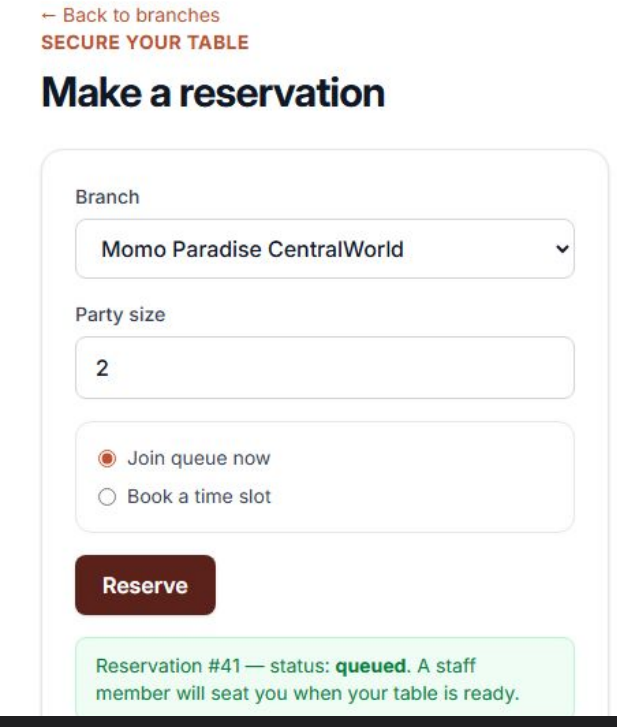
```
fn_login_staff('branch2.manager', 'pw')
```

→ staff row incl. role → app routes to the staff back office

```
SELECT * FROM fn_login_customer( p_email 'demo.presenter@example.com', p_password 'password123');
```

	customer_id	name	email	phone
1	63	Demo Presenter	demo.presenter@example.com	0812345678

Screens → Functions: Browse & Reserve *home, branch detail, reservation form*




```
fn_get_branches()
```

→ all 10 branches, alphabetical — the home-screen branch picker

```
fn_get_available_tables(1, 4)
```

→ free tables at branch 1 seating ≥ 4, smallest fit first

```
fn_create_reservation(1,1,2,'2026-07-11 18:00',4)
```

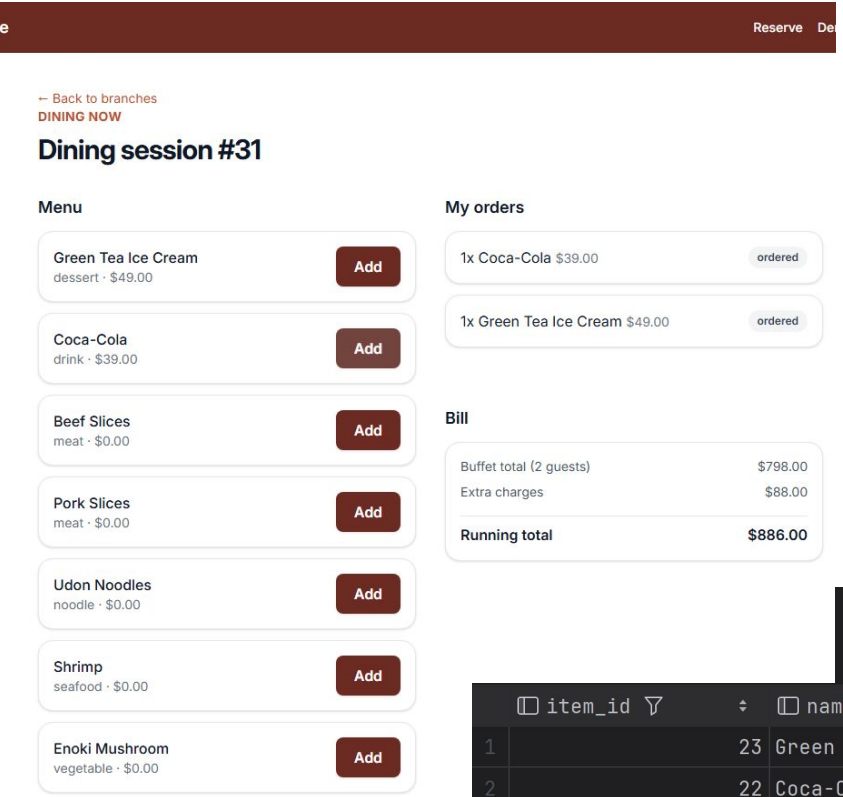
→ timed booking → status 'reserved', returns reservation_id

```
fn_create_reservation(3, 2, NULL, NULL, 2)
```

→ no time_slot → walk-in queue entry, status ‘queued’


```
SELECT * FROM fn_create_reservation( p_customer_id 63, p_branch_id 2, p_table_id NULL, p_slot_time NULL, p_party_size 2);
```

Screens → Functions: In-Session Ordering *the tier rule in action*




```
fn_get_tier_menu(2)
```

→ only dishes in this session's tier (via TIER_MENU_ITEM join)

```
fn_place_order(2, 3, 2)
```

→ inserts order line, snapshots current price into unit_price

```
fn_place_order(4, 3, 1)
```

→ *ERROR: Item 3 is not included in this session's tier*

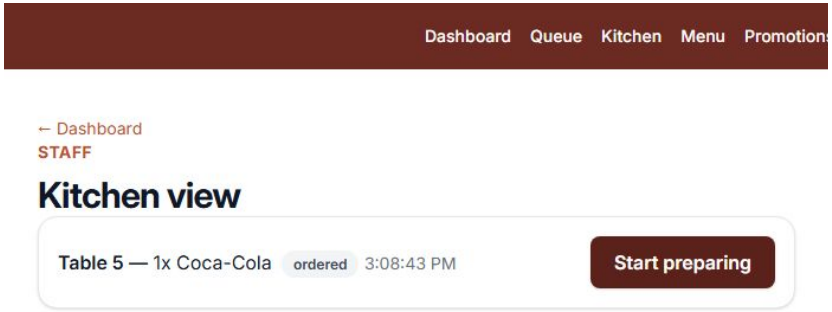
```
SELECT * FROM fn_get_tier_menu( p_session_id 31);
```

	item_id	name	category	price	available
1	23	Green Tea Ice Cream	dessert	49	true
2	22	Coca-Cola	drink	39	true
3	16	Beef Slices	meat	0	true

7 rows

Screens → Functions: Staff — Queue & Kitchen

seating and the live order feed




```
fn_get_queue(2)
```

→ waiting parties at branch 2, oldest first

```
fn_seat_reservation(7, 5, 2)
```

→ marks reservation 'seated', table occupied — ready to open a session

```
fn_get_kitchen_orders(2)
```

→ unserved dishes for active sessions, oldest first (indexed)

```
fn_update_order_status(15, 'served')
```

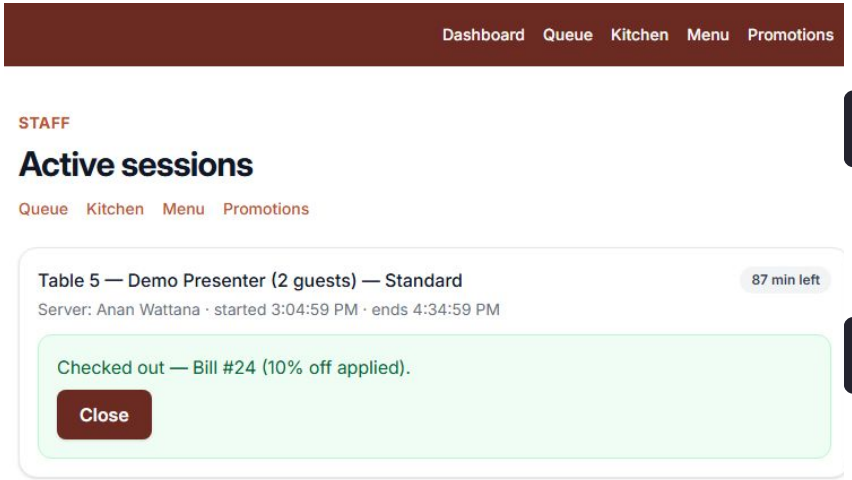
→ kitchen marks a dish done — disappears from the feed

```
SELECT * FROM fn_get_kitchen_orders( p_branch_id 2);
```

	order_line_id	session_id	table_id	item_name	quantity	ordered_at
1	50	33	5	Coca-Cola	1	2026-07-10 19:07:12.4447

Screens → Functions: Checkout & Points *one transaction*

closes the visit




```
fn_get_current_bill(1)
```

→ running total: tier price × guests + Σ(qty × unit_price)

```
fn_checkout(1,'WELCOME10','credit_card')
```

→ ONE transaction: bill + discount + points earned + session closed + table freed; row-locked (FOR UPDATE) against double checkout

```
fn_get_point_history(3)
```

→ the earn shows up on the customer's profile immediately

```
SELECT * FROM fn_get_bill(24);
```

bill_id	session_id	customer_id	customer_name	branch_name
24	31	63	Demo Presenter	Momo Paradise CentralWorld

extra_charges	discount_amount	promotion_code	final_total	payment_method
88	88.6	WELCOME10	797.4	cash

Live Demo

one full visit, end to end

1. Register & log in
2. Reserve / join queue
3. Staff seats the party & opens a session
4. Order dishes — watch a wrong-tier order get rejected
5. Kitchen serves
6. Checkout: bill + coupon + points
7. momo_app vs. tables: permission denied

Summary

14	28	25	3
tables, 3NF	SQL functions	API endpoints	security layers

All business rules live in the database — the app is a thin, least-privilege client.

Live database on Render · github.com/taro-5453/database-webapp

Thank you